# Основы

Statements and expressions, variables and assignment

# Общие слова о C++

# C++

C++ - это компилируемый язык общего назначения.

C++ был создан в начале 80-х годов Бьёрном Страуструпом на основе языка C, чтобы добавить объектно-ориентированные возможности. Хотя C++ является наследником языка C и заимствует многие его конструкции, C не является подмножеством C++ - не всякий код C компилируется в рамках C++.

## Три отличительные особенности языка C++

- Объектно-ориентированные возможности
- Обобщённое программирование
- Автоматический детерминированный деструктор

# C++

C++ включает множество разнообразных концепций

- Типизация
- Ссылки
- Указатели (для прямой работы с памятью)
- Перегрузка функций и операторов
- Статическая и динамическая память
- Типы, определяемые пользователем (enum, struct, union)
- Классы и методы
- Автоматические деструкторы
- Управление копированием и перемещением
- Шаблоны функций и классов
- Пространства имён
- Лямбда-выражения
- Асинхронные вызовы и корутины

# Как получаются программы на C++?

Исходный код на языке C++ транслируется в машинный код, проходя ряд преобразований.



# Первая программа

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Hello World!" << std::endl;
5      return 0;
6  }
```

Директива для препроцессора, вставка заголовка

Основная функция

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Hello World!" << std::endl;
5      return 0;
6  }
```

Литеральная константа

Выражение для вывода на экран

Код возврата

# Базовые концепции

statement и expression

# Statement

Исполняемой единицей кода на языке C++ является «*statement*» - некоторая инструкция. Существует строгий порядок исполнения инструкций: инструкции C++ исполняются слева направо и сверху вниз. Каждая инструкция завершается символом «*;*».

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Hello"; std::cout << " "; std::cout << "World!";
5      return 0;
6  }
```

```
1  #include <iostream>
2
3  int main() {
4      std::cout << "Hello";
5      std::cout << " ";
6      std::cout << "World!";
7      return 0;
8  }
```

Два представленных фрагмента кода делают одно и то же: выводят в консоль сообщение «Hello World!». Однако они различаются *форматированием*, т.е. расположением отдельных инструкций. Форматирование в C++ не влияет на порядок исполнения кода.

# Expression

«*Expression*» - это некоторое вычисляемое выражение. Основные отличия expression от statement:

1. expression обязательно вычисляется к некоторому значению, хотя в некоторых ситуациях это значение может оказаться пустым (void);
2. порядок исполнения отдельных частей выражения может быть до некоторой степени изменён в целях оптимизации кода.

```
std::cout << "Hello" << " " << "World!"
```




```
(((std::cout << "Hello") << " ") << "World!")
```

Порядком исполнения операторов можно управлять с помощью круглых скобок.

* данное *expression* вычисляется к *std::cout*, что позволяет в одном выражении вывести сразу много различных фрагментов данных, не дробя их на отдельные statement.

# Expression

Форматирование также не влияет на expression:

```
3   int main() {
4       std::cout
5           << "Hello"
6           << " "
7               << "World!";
8   }
```

# Комментарии

*Комментариями* называется любой текст, который заключён в специальные управляющие конструкции: /* и */ для многострочных комментариев и // для однострочных. *Комментарии не проходят компиляцию и игнорируются. Комментарии следует оставлять для пояснений в коде.*

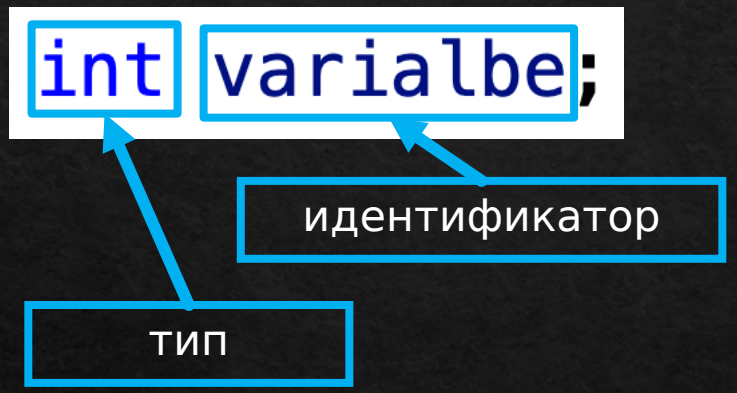
```
1  #include <iostream>
2
3  int main() {
4      std::cout << "This line is going to be printed" << '\n';
5      // std::cout << "This line is commented and ignored" << '\n';
6
7      /*std::cout
8          << "This"
9          << "Multiline"
10         << "Block"
11         << "Is"
12         << "Also"
13         << "Ignored";*/
14
15     std::cout << "This line is still be printed out" << '\n';
16 }
```

# Переменные

Объявление переменной (declaration), инициализация (initialization), присваивание (assignment)

# Переменная

В языке C++ *переменной* называется именованная область памяти, с которой связаны тип (*type*) хранимого значения и идентификатор (*identifier*).




*Тип* необходим для определения количества памяти, которое переменная занимает и правильной интерпретации этих данных. *Идентификатор* или имя переменной необходим для обращения к памяти, чтобы читать или записывать в эту память данные.

Тип может быть либо предопределён компилятором, либо определён сторонним кодом (например стандартной библиотекой), либо определён самим программистом, но обязательно заранее, до объявления переменной.

Идентификатор определяется вместе с переменной. Идентификатор должен удовлетворять следующим условиям: он может состоять только из латинских букв, цифр и знака нижнего подчёркивания, идентификатор не может начинаться с цифры.

# Объявление переменной

```
int a;
int b;
```

```
int A;
int B;
```

Несколько вариантов объявления переменных с верными идентификаторами. *Отметим, что заглавная и строчная буквы задают разные идентификторы!*

```
int a, b;

int _varialbe1, _variable2;

int variable_with_a_very_long_name;
```

* все описанные выше переменные объявлены с типом *int*, который состоит из 4 байтов на 32х и 64х битных системах и допускает хранение и арифметические операции над целыми числами в диапазоне от -2^31 до 2^31 - 1.

# Инициализация

*Инициализацией* переменной называется указание значения переменной при её объявлении.

```
int a = 3;

int a = 3, b = 5;

int a, b = 5;
```

```
int a(3);
int b(3);

int a(3), b(5), c(7);
```

```
int a = 3, b = a;


int c(a), d(b);
```

Если переменной при объявлении не присвоено никакого значения, то она называется *неинициализированной*.

* неинициализированные переменные опасны, им можно давать значения с использованием приставаивания, но нельзя читать значения, которые в них хранятся.

# Присваивание

*Присваиванием* называется изменение значения переменной, которая была ранее объявлена и, возможно, инициализирована.

```
3   int main() {
4       int a;
5       // std::cout << a;
6
7       a = 3;
8       std::cout << a;
9   }
```

объявление неинициализированной переменной

недопустимая операция, переменная не содержит значения

присваивание заменяет значение в переменной

теперь в переменной лежит конкретное значение, операция допустима

# Константы

Объявление переменной (declaration), инициализация (initialization), присваивание (assignment)

# Объявление констант

*Константы* не меняют своего значения после того, как были инициализированы.

```
int const cv_value = 5;
const int another_cv_value = 7;
```

При объявлении слева или справа от имени типа следует указать ключевое слово const.

Можно объявить несколько констант в одном statement.

```
int const cv_a = 1, cv_b = 2, cv_c = 3;
```

Константе ничего нельзя присвоить:

```
cv_c = 2;
```

```
main.cpp:9:10: error: cannot assign to variable 'cv_c' with const-qualified type 'const int'
    cv_c = 2;
    ~~~~ ^
```

Константа должна быть инициализирована:

```
const int cv;
```

```
main.cpp:11:15: error: default initialization of an object of const type 'const int'
    const int cv;
```

# Литеральные константы

*Литеральными константами* называются записанные литералами значения в коде. Они не имеют «имён». Сама запись кодирует значение.

```
1  #include <iostream>
2
3  int main() {
4      char ch = 't';
5
6      int a = 78945;
7
8      std::cout << "Hello World!" << std::endl;
9
10     return 0;
11 }
```



# Базовые типы

# Базовые типы

```
// символьный тип
char // 1 байт

// логический тип
bool // 1 байт

// целочисленный тип
unsigned / signed char              // 1 байт
unsigned / signed short             // 2 байта
unsigned / signed int               // 2/4 байта
unsigned / signed long int          // 4/8 байт
unsigned / signed long long int     // 4/8 байт

// действительные числа
float           // 4 байта
double          // 8 байт
long double     // 8/16 байт
```

```
short sh;      // signed short
int i;         // signed int
long l;        // signed long int
long long ll; // signed long long int
unsigned u;    // unsigned int
```

Чтобы узнать размер типа в байтах можно использовать оператор sizeof.

```
int i;
std::cout <<
    sizeof(i) << " " << sizeof(int)
    << std::endl;
```

# Максимальные и минимальные значения

```
std::numeric_limits<int>::max();
std::numeric_limits<long long>::min();
std::numeric_limits<double>::epsilon();
```

Максимальное значение

Минимальное значение

Машинное эпсилон

```
#include <iostream>
#include <limits>

int main() {

    std::cout <<
        std::numeric_limits<long long>::min() << '\n' <<
        std::numeric_limits<long long>::max() << '\n' <<
    std::endl;

    return 0;
}
```

```
-9223372036854775808
9223372036854775807
```

# Ключевое слово auto

auto позволяет не писать тип переменной или константы явно, если тип может быть выведен при инициализации.

```
int a = 2, b = 3, c = 9;
auto exp = (a + b) * (b + c);
```

```
auto ch = 'h';
auto t = true;
auto i = 1;
auto u = 1u;
auto l = 1l;
auto ll = 1ll;
auto ul = 1ul;
auto ull = 1ull;
auto f = 1.f;
auto d = 1.;
```

Использование суффиксов для указания типов литеральных констант.

- u/U – unsigned int
- ul/UL – unsigned long int
- ull/ULL – unsigned long long int
- f – float
- нет суффикса и число без десятичной точки – int
- нет суффикса и число с десятичной точкой - double